LAPORAN PRAKTIKUM
ALGORITMA & STRUKTUR DATA
MODUL 2

Stack dan Queue

Oleh:
Laras Kurniadesi NIM. 2510817120010

PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
APRIL
2026

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM ALGORITMA & STRUKTUR DATA
MODUL 2

Laporan Praktikum Algoritma & Struktur Data Modul 2: Stack dan Queue ini disusun sebagai syarat lulus mata kuliah Praktikum Algoritma & Struktur Data. Laporan Prakitkum ini dikerjakan oleh:

Nama Praktikan : Laras Kurniadesi
NIM : 2510817120010

Menyetujui, Asisten Praktikum	Mengetahui, Dosen Penanggung Jawab Praktikum
Muhammad Fauzan Ahsani NIM. 2310817310009	Ir. Muhamamd Alkaff, S,Kom, M.Kom, Ph.D NIP. 198606132015041001

DAFTAR ISI

LEMBAR PENGESAHAN 2
DAFTAR ISI 3
DAFTAR GAMBAR 4
DAFTAR TABEL 5
SOAL 1 6
A. Source Code 6
B. Pembahasan 7
SOAL 2 8
A. Source Code 9
B. Output Program 10
C. Pembahasan 10
SOAL 3 11
A. Source Code 11
B. Pembahasan 12
SOAL 4 13
A. Source Code 14
B. Output Program 14
C. Pembahasan 15
TAUTAN GIT 16

DAFTAR GAMBAR

Gambar 1. Screenshot Hasil Jawaban Soal 1B 10
Gambar 2. Screenshot Hasil Jawaban Soal 2B 10
Gambar 3. Screenshot Hasil Jawaban Soal 3B 10
Gambar 4. Screenshot Hasil Jawaban Soal 1D 14
Gambar 5. Screenshot Hasil Jawaban Soal 2D 15
Gambar 6. Screenshot Hasil Jawaban Soal 3D 15

DAFTAR TABEL

Table 1. Source Code stack.cpp 6
Table 2. Source Code prob1.cpp 9
Table 3. Source Code queue.cpp 11
Table 4. Source Code prob2.cpp 14

SOAL 1

Pada file `stack.h` yang diberikan, implementasikan seluruh fungsi yang dideklarasikan ke dalam file `stack.cpp`. Untuk menangani kondisi tidak valid (invalid case), gunakan mekanisme exception handling dengan `throw`.

Jelaskan alur algoritma dari setiap fungsi yang diimplementasikan, serta berikan justifikasi atau pembuktian singkat mengenai kebenaran algoritma tersebut. Selain itu, lakukan analisis kompleksitas waktu dan ruang untuk setiap fungsi yang ada. (Gunakan Big-O Notation).

A. Source Code

Table 1. Source Code stack.cpp

```
#include "stack.h"
#include <stdexcept>

void init(Stack* s) {
    s->top = s->data - 1;
}

bool isEmpty(const Stack* s) {
    return s->top == s->data - 1;
}

bool isFull(const Stack* s) {
    return s->top == s->data + MAX - 1;
}

void push(Stack* s, int value) {
    if (isFull(s)) {
        throw std::overflow_error("Stack penuh");
    }
    s->top++;
    *(s->top) = value;
}

void pop(Stack* s) {
    if (isEmpty(s)) {
        throw std::underflow_error("Stack kosong");
    }
    s->top--;
}

```

```
int peek(const Stack* s) {
    if (isEmpty(s)) {
        throw std::underflow_error("Stack kosong");
    }
    return *(s->top);
}
```

B. Pembahasan

Program ini dibuat untuk mengimplementasikan struktur data stack menggunakan array dengan bantuan pointer sebagai penunjuk posisi elemen paling atas. Cara kerjanya dimulai dari fungsi *init* yang mengatur kondisi awal stack agar kosong dengan mengarahkan pointer *top* ke sebelum elemen pertama. Selanjutnya, fungsi *push* digunakan untuk menambahkan data ke dalam stack dengan menaikkan posisi *top*, sedangkan fungsi *pop* digunakan untuk menghapus data paling atas dengan menurunkan posisi *top*. Untuk melihat nilai teratas tanpa menghapusnya, digunakan fungsi *peek*. Selain itu, terdapat fungsi *isEmpty* dan *isFull* yang berfungsi untuk mengecek apakah stack dalam kondisi kosong atau penuh sebelum operasi dilakukan, sehingga dapat menghindari error dengan menggunakan exception handling. Secara keseluruhan, program ini sudah menerapkan konsep stack dengan prinsip Last In First Out (LIFO) dengan baik dan efisien.

SOAL 2

Si Palui menemukan sebuah mesin hitung tua di gudang. Mesin tersebut memiliki cara kerja yang tidak biasa. Alih-alih menggunakan format perhitungan seperti yang biasa kita kenal, mesin ini membaca input berupa deretan simbol yang dipisahkan oleh spasi.

Setiap simbol bisa berupa:
• sebuah bilangan bulat, atau
• sebuah operator (+, -, *, /)

Cara kerja mesin adalah sebagai berikut:
• Mesin membaca simbol dari kiri ke kanan.
• Jika simbol adalah bilangan, maka bilangan tersebut disimpan ke dalam memori mesin.
• Jika simbol adalah operator, maka mesin akan mengambil dua nilai yang paling baru dimasukkan ke dalam memori yang disimpan, melakukan operasi sesuai operator tersebut, lalu menyimpan kembali hasilnya ke dalam memori.

Setelah seluruh simbol diproses, akan tersisa satu nilai di dalam memori. Nilai inilah yang ditampilkan sebagai hasil akhir.

Sebagai asisten Si Palui, tugas Anda adalah membantu menghitung hasil dari mesin tersebut.

Batasan
• $1 \leq n \leq 100$
• $-1000 \leq Ai \leq 1000$
Keterangan:
• n adalah jumlah simbol dalam ekspresi
• Ai adalah nilai bilangan pada input
• Simbol yang valid hanya:
‣ bilangan bulat
‣ operator: +, -, *, /

Format Input

```
n
A1 A2 ... An
```

Keterangan:

• Baris pertama n menyatakan jumlah atau panjang simbol

• Baris kedua berisi simbol A sebanyak n buah

Format Output

Sebuah bilangan hasil dari kalkulator Si Palui.

Contoh Kasus

Input	Output
3 2 3 +	5
9 5 1 2 + 4 * + 3 -	14
5 10 2 / 3 *	15

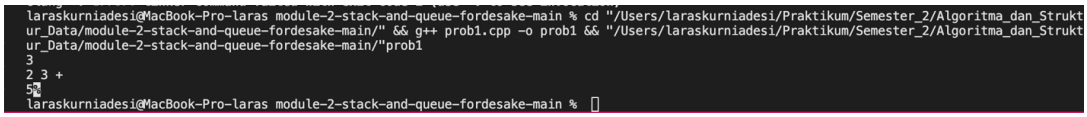
A. Source Code

Table 2. Source Code prob1.cpp

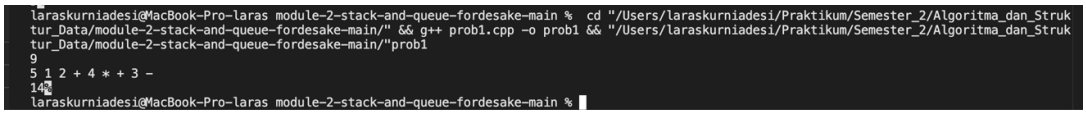
```
#include <iostream>
#include <stack>
using namespace std;

int main() {
    int n;
    cin >> n;

    stack<int> s;

    for (int i = 0; i < n; i++) {
        string x;
        cin >> x;

        if (isdigit(x[0]) || (x[0] == '-' && x.length()
> 1)) {
            s.push(stoi(x));
        }
        else {
            int b = s.top(); s.pop();
            int a = s.top(); s.pop();

```

```
            if (x == "+") s.push(a + b);
            else if (x == "-") s.push(a - b);
            else if (x == "*") s.push(a * b);
            else if (x == "/") s.push(a / b);
        }
    }

    cout << s.top();
    return 0;
}
```

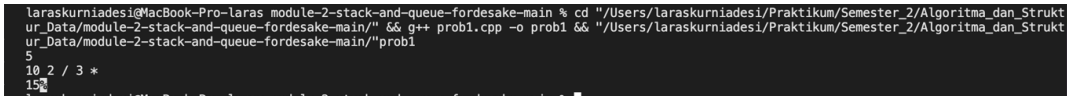
B. Output Program



Gambar 1. Screenshot Hasil Jawaban Soal 1B



Gambar 2. Screenshot Hasil Jawaban Soal 2B



Gambar 3. Screenshot Hasil Jawaban Soal 3B

C. Pembahasan

Program ini digunakan untuk menghitung hasil dari suatu ekspresi matematika yang ditulis dalam bentuk postfix (Reverse Polish Notation) dengan memanfaatkan struktur data stack. Cara kerjanya, program pertama membaca jumlah symbol n, lalu membaca setiap simbol satu per satu. Jika simbol berupa angka, maka langsung dimasukkan ke dalam stack. Namun jika simbol berupa operator seperti $+$, $-$, $*$, atau $/$, maka program akan mengambil dua angka terakhir yang ada di dalam stack, melakukan operasi sesuai operator tersebut, lalu menyimpan kembali hasilnya ke dalam stack. Proses ini dilakukan terus sampai semua simbol selesai diproses, dan pada akhirnya hanya tersisa satu nilai di dalam stack yang merupakan hasil akhir perhitungan. Dengan pendekatan ini, perhitungan bisa dilakukan secara efisien tanpa perlu memperhatikan prioritas operator seperti pada notasi biasa.

SOAL 3

Pada file queue.h yang diberikan, implementasikan seluruh fungsi yang dideklarasikan ke dalam file queue.cpp. Untuk menangani kondisi tidak valid (invalid case), gunakan mekanisme exception handling dengan throw.

Jelaskan alur algoritma dari setiap fungsi yang diimplementasikan, serta berikan justifikasi atau pembuktian singkat mengenai kebenaran algoritma tersebut. Selain itu, lakukan analisis kompleksitas waktu dan ruang untuk setiap fungsi yang ada. (Gunakan Big-O Notation).

A. Source Code

Table 3. Source Code queue.cpp

```
#include "queue.h"
#include <stdexcept>

void init(Queue* q) {
    q->front = q->data;
    q->rear = q->data - 1;
}

bool isEmpty(const Queue* q) {
    return q->rear < q->front;
}

bool isFull(const Queue* q) {
    return q->rear == q->data + MAX - 1;
}

void enqueue(Queue* q, int value) {
    if (isFull(q)) {
        throw std::overflow_error("Queue penuh");
    }
    q->rear++;
    *(q->rear) = value;
}

void dequeue(Queue* q) {
    if (isEmpty(q)) {
        throw std::underflow_error("Queue kosong");
    }
    q->front++;
```

```
}

int front(const Queue* q) {
    if (isEmpty(q)) {
        throw std::underflow_error("Queue kosong");
    }
    return *(q->front);
}

int back(const Queue* q) {
    if (isEmpty(q)) {
        throw std::underflow_error("Queue kosong");
    }
    return *(q->rear);
}
```

B. Pembahasan

Program ini digunakan untuk mengimplementasikan struktur data queue menggunakan array dengan bantuan pointer sebagai penunjuk bagian depan (front) dan belakang (rear). Pada awalnya, fungsi `init` mengatur kondisi queue agar kosong dengan mengarahkan `front` ke awal array dan `rear` ke sebelum elemen pertama. Fungsi `enqueue` digunakan untuk menambahkan data ke bagian belakang queue dengan menaikkan posisi `rear`, sedangkan fungsi `dequeue` digunakan untuk menghapus data dari bagian depan dengan menaikkan posisi `front`. Untuk melihat nilai tanpa menghapusnya, digunakan fungsi `front` untuk elemen depan dan `back` untuk elemen belakang. Selain itu, terdapat fungsi `isEmpty` dan `isFull` yang berfungsi untuk mengecek kondisi queue sebelum operasi dilakukan, sehingga dapat mencegah error seperti mengambil data dari queue kosong atau menambahkan data saat queue penuh dengan menggunakan exception handling. Secara keseluruhan, program ini sudah menerapkan konsep queue dengan prinsip First In First Out (FIFO) dengan baik dan efisien.

SOAL 4

Si Palui memiliki sebuah deretan angka yang tersusun rapi. Ia tertarik untuk mengamati sebagian angka secara berurutan dengan jumlah tertentu.

Ia memilih sejumlah k angka yang bersebelahan, lalu menghitung jumlahnya. Setelah itu, ia menggeser pilihannya satu langkah ke kanan dan kembali menghitung jumlah dari angka angka yang terpilih.

Proses ini dilakukan terus hingga tidak memungkinkan lagi untuk memilih k angka berurutan. Sebagai asistennya, bantulah Si Palui untuk menentukan hasil perhitungan tersebut.

Batasan

• $1 \leq n \leq 100$

• $-1 \leq k \leq n$

• $-1000 \leq Ai \leq 1000$

Keterangan:

• n: jumlah elemen

• k: ukuran jendela

• Ai: elemen array

Format Input

n k

`A1 A2 ... An`

Keterangan:

• Baris pertama n menyatakan jumlah atau panjang array. Lalu k besar atau jumlah elemen dalam satu kelompok ayng dibentuk

• Baris kedua berisi simbol `A` sebanyak n buah

Format Output

Cetak hasil tiap masing-masing kelompok dalam satu baris dipisahkan dengan spasi

Contoh Kasus

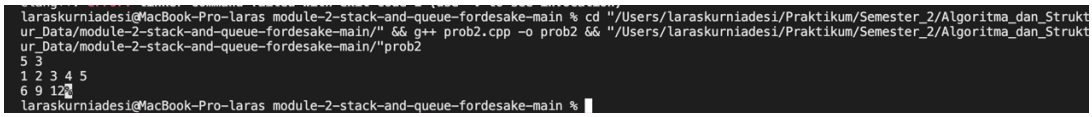
Input	Output
5 3 1 2 3 4 5	6 9 12
5 5 1 2 3 4 5	15
6 2 4 -1 2 -3 5 1	3 1 -1 2 6

A. Source Code

Table 4. Source Code prob2.cpp

```
#include <iostream>
using namespace std;

int main() {
    int n, k;
    cin >> n >> k;

    int arr[100];
    for (int i = 0; i < n; i++) {
        cin >> arr[i];
    }

    int sum = 0;
    for (int i = 0; i < k; i++) {
        sum += arr[i];
    }

    cout << sum;

    for (int i = k; i < n; i++) {
        sum = sum - arr[i - k] + arr[i];
        cout << " " << sum;
    }

    return 0;
}
```

B. Output Program



Gambar 4. Screenshot Hasil Jawaban Soal 1D

```
laraskurniadesi@MacBook-Pro-laras module-2-stack-and-queue-fordesake-main % cd "/Users/laraskurniadesi/Praktikum/Semester_2/Algoritma_dan_Strukt
ur_Data/module-2-stack-and-queue-fordesake-main/" && g++ prob2.cpp -o prob2 && "/Users/laraskurniadesi/Praktikum/Semester_2/Algoritma_dan_Strukt
ur_Data/module-2-stack-and-queue-fordesake-main/"prob2
5 5
1 2 3 4 5
15%
laraskurniadesi@MacBook-Pro-laras module-2-stack-and-queue-fordesake-main %
```

Gambar 5. Screenshot Hasil Jawaban Soal 2D

```
5 5
1 2 3 4 5
15%
laraskurniadesi@MacBook-Pro-laras module-2-stack-and-queue-fordesake-main % cd "/Users/laraskurniadesi/Praktikum/Semester_2/Algoritma_dan_Strukt
ur_Data/module-2-stack-and-queue-fordesake-main/" && g++ prob2.cpp -o prob2 && "/Users/laraskurniadesi/Praktikum/Semester_2/Algoritma_dan_Strukt
ur_Data/module-2-stack-and-queue-fordesake-main/"prob2
6 2
4 -1 2 -3 5 1
3 1 -1 2 6%
laraskurniadesi@MacBook-Pro-laras module-2-stack-and-queue-fordesake-main %
```

Gambar 6. Screenshot Hasil Jawaban Soal 3D

C. Pembahasan

Program ini digunakan untuk menghitung jumlah dari beberapa elemen yang berurutan dalam sebuah array menggunakan metode sliding window. Cara kerjanya dimulai dengan membaca input jumlah data n dan ukuran window k, lalu menyimpan elemen ke dalam array. Setelah itu, program menghitung jumlah k elemen pertama sebagai window awal dan langsung menampilkannya. Selanjutnya, window digeser satu per satu ke kanan dengan cara mengurangi nilai elemen yang keluar dari window dan menambahkan elemen baru yang masuk, sehingga tidak perlu menghitung ulang dari awal. Hasil dari setiap pergeseran window tersebut kemudian ditampilkan secara berurutan. Dengan cara ini, proses perhitungan menjadi lebih efisien karena hanya melakukan update sederhana pada setiap langkah, bukan menjumlahkan ulang seluruh elemen di dalam window.

TAUTAN GIT

https://github.com/fordesake/stack-and-queue.git